

Contents

Guia do Músico — Open IEM Platform	1
1. Introdução	1
2. Requisitos de Sistema	1
Servidor (quem opera o sistema)	1
Músico (cliente)	2
3. Preparação do Servidor	2
3.1 Pré-requisitos	2
3.2 Clone e Build	2
3.3 Gerar Chaves JWT	2
3.4 Iniciar com Docker Compose (desenvolvimento)	3
4. Criação de Usuário Músico	3
5. Login no Sistema	3
6. Interface do Músico (Musician PWA)	3
Estados de Conexão	3
Controles Disponíveis	3
7. Mix e Canais	4
8. Áudio WebRTC — △ SIMULATED	4
9. Permissões por Papel	4
10. Diagnóstico	5
Verificar saúde do servidor	5
Logs do servidor	5
Problemas Comuns	5
11. Rede LAN	5
12. Segurança	5
Mix atribuído e controles	6
13. Limitações Conhecidas	6
14. Fluxo de Configuração Rápida	6

Guia do Músico — Open IEM Platform

Versão: Phase 7 · **Idioma:** pt-BR
Última atualização: 2026-09-09

1. Introdução

Open IEM Platform é uma plataforma de monitoramento in-ear de código aberto para bandas, igrejas, salas de ensaio e produção ao vivo. O servidor recebe áudio multicanal de uma interface de áudio ou mesa digital, cria mixagens independentes por músico e permite controle em tempo real via navegador web.

Este guia é voltado ao **músico** — o usuário que controla seu próprio mix de monitoramento.

2. Requisitos de Sistema

Servidor (quem opera o sistema)

Componente	Requisito
SO alvo	Linux ARM64 (Raspberry Pi 5) ou x86_64
Áudio	PipeWire + ALSA (hardware real)

Componente	Requisito
Rust	stable 1.78+
Node.js	22+ (para interface web)
Docker	24+ (opcional, para dev)

SIMULATED: No VPS de desenvolvimento, o caminho de áudio real (PipeWire) não está disponível. Sinalização WebRTC e controles funcionam; áudio requer Raspberry Pi 5 com PipeWire.

Músico (cliente)

Componente	Requisito
Navegador	Chrome 120+ ou Firefox 120+ com WebRTC
Rede	LAN (HTTP local) ou HTTPS quando exposto externamente
SO cliente	Qualquer — não testado em Windows/macOS (PENDENTE)

3. Preparação do Servidor

3.1 Pré-requisitos

```
# Rust stable
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup update stable
```

```
# Node.js 22
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo bash -
sudo apt-get install -y nodejs
```

```
# Docker (opcional)
# Consulte https://docs.docker.com/engine/install/
```

3.2 Clone e Build

```
git clone https://github.com/rickslamaral/open-iem-platform.git
cd open-iem-platform
```

```
# Build servidor Rust
cd server
cargo build --release
cd ..
```

```
# Instalar deps da interface do músico
cd web/musician
npm ci
```

3.3 Gerar Chaves JWT

O servidor usa criptografia Ed25519 para tokens JWT. Gerar antes de iniciar:

```
mkdir -p secrets
```

```
# Chave privada (NUNCA commitar)
```

```
openssl genpkey -algorithm ed25519 -out secrets/jwt_private.pem
```

Chave pública

```
openssl pkey -in secrets/jwt_private.pem -pubout -out secrets/jwt_public.pem
```

⚠ **Segurança:** secrets/ está no .gitignore. Nunca versionar chaves privadas.

3.4 Iniciar com Docker Compose (desenvolvimento)

```
docker compose up --build
```

Serviços disponíveis: - API: http://localhost:3000 - Musician PWA: http://localhost:5173 - Engineer UI: http://localhost:5174

4. Criação de Usuário Músico

PENDENTE: Interface administrativa para criar usuários via CLI ou painel web ainda não implementada. Crie usuários diretamente via API (requer token ADMIN):

Exemplo - requer token de administrador no header Authorization

```
curl -X POST http://localhost:3000/api/v1/admin/users \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <admin_token>" \
-d '{"username": "joao", "password": "senha-segura", "role": "MUSICIAN"}'
```

Nota: A rota /api/v1/admin/users está **PENDENTE** de implementação. Consulte o engenheiro de sistema para criação de contas no estado atual.

5. Login no Sistema

1. Abra o navegador e acesse http://<ip-do-servidor>:5173
2. Informe usuário e senha cadastrados pelo administrador
3. Clique em **Entrar**

O que acontece internamente: - POST /api/v1/auth/login retorna access_token (armazenado em memória — nunca no localStorage) - Cookie refresh_token httpOnly é definido automaticamente pelo servidor - O token de acesso expira em 15 minutos; o refresh é renovado automaticamente

6. Interface do Músico (Musician PWA)

Estados de Conexão

Status	Significado
Conectando	WebSocket sendo estabelecido
Conectado	Controle em tempo real ativo
Desconectado	Sem conexão — reconexão automática

Controles Disponíveis

- **Faders de canal (1-8):** ajuste de ganho em dB por canal
- **Mute por canal:** silenciar canal individual

- **Volume master:** ganho geral do mix (*implementado na UI; wire-up com o servidor previsto para fase seguinte*)
- **Logout:** encerra sessão e desconecta WebSocket

7. Mix e Canais

Cada músico tem acesso ao seu mix independente. Os controles enviam mensagens para o servidor via:

REST (channel control):

```
PUT /api/v1/channels/:index
Content-Type: application/json
Authorization: Bearer <token>
```

```
{ "gain_db": -3.0 }      # ajuste de ganho
{ "muted": true }       # mute
```

WebSocket (/ws/v1):

```
{ "type": "SetChannelGain", "data": { "channel": 2, "gain_db": -6.0 } }
{ "type": "SetChannelMute", "data": { "channel": 2, "muted": false } }
```

Faixas válidas: gain_db entre -144.0 dBFS e +12.0 dBFS.

8. Áudio WebRTC — SIMULATED

A sinalização WebRTC está implementada (oferta SDP, resposta, candidatos ICE via str0m). No entanto:

Componente	Estado
Sinalização SDP/ICE	☐ Implementado
Fluxo de áudio Opus	 SIMULATED — frames de silêncio de 20 ms
PipeWire real	☐ Requer Raspberry Pi 5 com PipeWire
DTLS/SRTP transport	 SIMULATED no VPS

Para áudio real funcionar, o servidor precisa rodar em hardware Raspberry Pi 5 com PipeWire instalado e interface de áudio USB conectada.

9. Permissões por Papel

Ação	MUSICIAN	ENGINEER	ADMIN
Ajustar próprio mix	☐	☐	☐
Ver sessões de áudio de todos	☐	☐	☐
Gerenciar usuários	☐	☐	☐
Acessar mix de outro músico	☐	☐	☐
WebSocket controle	☐	☐	☐

Nota: Enforcement de propriedade de mix por músico está **PENDENTE** (depende do modelo de designação de mixes — Phase 8).

10. Diagnóstico

Verificar saúde do servidor

```
curl http://localhost:3000/health  
# Resposta esperada: 200 OK
```

Logs do servidor

```
RUST_LOG=open_iem=debug cargo run --release  
# Ou com Docker:  
docker compose logs -f api-server
```

Problemas Comuns

Sintoma	Causa Provável	Solução
401 Unauthorized	Token expirado	Fazer logout e login novamente
403 Forbidden	Papel insuficiente	Verificar papel do usuário com admin
422 Unprocessable Entity	Campo de role com casing errado	Usar SCREAM-ING_SNAKE_CASE: "MUSICIAN"
WebSocket desconecta	Rede instável	Reconexão automática ocorre
CORS error	Acesso de origem diferente	Usar mesmo host/porta

11. Rede LAN

Para acesso de outros dispositivos na mesma rede:

```
# Descobrir IP do servidor  
ip addr show | grep "inet " | grep -v 127.0.0.1  
  
# Iniciar servidor escutando em todas as interfaces (dev only)  
OPEN_IEM_BIND=0.0.0.0:3000 OPEN_IEM_DEV_ALLOW_NON_LOOPBACK=true cargo run  
Músicos na mesma rede acessam: http://<ip-do-servidor>:5173
```

12. Segurança

Esta seção descreve o modelo de segurança atual e os requisitos antes de qualquer exposição externa.

- **HTTP local/LAN:** permitido somente em localhost e LAN isolada durante desenvolvimento.
- **TLS obrigatório:** antes de expor o servidor externamente, configure Caddy ou Nginx como proxy reverso com TLS válido.
- **Tokens:** access token armazenado apenas em memória React (nunca localStorage/sessionStorage). Refresh token em cookie httpOnly.
- **Senhas:** Argon2 (nunca reversível). Nunca compartilhe sua senha.
- **Rate limiting e CSRF:** validação de header Origin ativa para WebSocket e formulários de auth.

Mix atribuído e controles

Cada músico pode receber um único mix pelo engenheiro/admin. Após atribuição, músico pode ler e alterar somente sends do próprio mix:

- GET /api/v1/mixes/{mix_idx}/sends/{ch_idx}
- PUT /api/v1/mixes/{mix_idx}/sends/{ch_idx}/gain
- PUT /api/v1/mixes/{mix_idx}/sends/{ch_idx}/pan
- PUT /api/v1/mixes/{mix_idx}/sends/{ch_idx}/mute

Sem atribuição ou tentando outro mix, API retorna 403 Forbidden. Controles de canal continuam restritos a Engineer/Admin. Áudio continua SIMULATED no VPS até validação real de PipeWire/Opus no Raspberry Pi 5.

13. Limitações Conhecidas

Item	Estado
Áudio real via PipeWire	❑ Requer Raspberry Pi 5 (SIMULATED em VPS)
Criação de usuário via UI	⚠ PENDENTE
HTTPS/TLS integrado	⚠ PENDENTE (usar proxy externo)
Windows/macOS (servidor)	⚠ Não testado
Designação de mix por músico	❑ Implementado no backend
Enforcement de ownership por MUSICIAN	❑ Implementado para sends e signaling; atomicidade de mutações em backlog
Volume master wire-up	⚠ PENDENTE

14. Fluxo de Configuração Rápida

Para testar o sistema agora:

1. Clone o repositório:

```
git clone https://github.com/rickslamaral/open-iem-platform.git && cd open-iem-platform
```

2. Gere as chaves JWT:

```
mkdir secrets
openssl genpkey -algorithm ed25519 -out secrets/jwt_private.pem
openssl pkey -in secrets/jwt_private.pem -pubout -out secrets/jwt_public.pem
```

3. Suba os serviços:

```
docker compose up --build
```

4. Aguarde os serviços ficarem saudáveis:

```
curl http://localhost:3000/health
```

5. Acesse a interface do músico: Abra http://localhost:5173 no navegador.

6. Faça login com as credenciais criadas pelo administrador.

7. Ajuste faders e mutes — as alterações são enviadas ao servidor em tempo real.

Lembre-se: Para áudio real, você precisa de Raspberry Pi 5 com PipeWire e uma interface de áudio USB. No VPS, apenas a sinalização e controles funcionam (SIMULATED).